Data Structures

Spring 2026

Tony Stone

Contents

1 Preface **3**
1.1 Mission Statement . . . 3
1.2 Acknowledgments . . . 3

2 Introduction's Introduction **4**
2.1 Python Examples . . . 4
2.1.1 I Hate Snake Case . . . 4
2.1.2 Further Reading . . . 4

3 Introduction to Data Structures **5**
3.1 Defining Data . . . 5
3.1.1 Atomic Data Types . . . 5
3.1.2 Abstract Data Types . . . 5
3.1.3 Primative Data Types . . . 5
3.1.4 Data Structures . . . 6
3.2 Moving Forward . . . 6

4 Basic Data Types and Structures **7**
4.1 Integer, Floating Point Numbers, Booleans . . . 7
4.1.1 Integer . . . 7
4.1.2 Floating Point Number . . . 7
4.1.3 Boolean . . . 7
4.2 Collection Types . . . 8
4.2.1 Strings . . . 8
4.2.2 Lists . . . 8
4.2.3 Maps . . . 8
4.2.4 Miscellaneous . . . 8
4.3 Moving forward . . . 9

5 Analysis **10**
5.1 Analyzing Algorithms . . . 10
5.2 Benchmarking . . . 10
5.3 Theoretical Analysis . . . 11
5.3.1 Intuition . . . 12

5.3.2 Unit of work . . . 12
5.3.3 Worst-case . . . 13
5.3.4 Analysis and Notation . . . 13
5.3.5 Big-O . . . 13

6 Linear Structures **15**
6.1 Stacks . . . 15
6.1.1 Stack implementation . . . 15
6.1.2 Stack Analysis . . . 16
6.2 Queue . . . 17
6.3 Queue Implementation . . . 17
6.3.1 Queue Analysis . . . 18
6.4 Deque . . . 18
6.4.1 Deque Implementation . . . 18
6.4.2 Deque Analysis . . . 18

7 Searching and Sorting **19**
7.1 Searching . . . 19
7.1.1 Sequential Search . . . 19
7.1.2 Binary Search . . . 19
7.2 Sorting . . . 20
7.2.1 Bubble Sort . . . 20
7.2.2 Merge Sort . . . 21

8 Trees **23**
8.1 Terms . . . 23
8.2 Types of Trees . . . 23
8.2.1 n-ary Trees . . . 23
8.2.2 Binary Trees . . . 23
8.2.3 Balanced Trees . . . 24
8.3 Binary Search Trees . . . 24
8.3.1 Lookup . . . 24

1 Preface

1.1 Mission Statement

With this work, I set out to develop a comprehensive set of lecture notes, on the topic of Data Structures. While developing these notes, I am keeping in mind both the lecturer, and the learner. My goal is that this will amount to an educational resource that acts as a guide throughout fundamental topics, in a manner which is accessible and painless for all parties. This work should provide confidence and direction to its reader, whether they are at the front of a classroom, or at their dining-room table.

1.2 Acknowledgments

I would like to thank the Center for Academic Success and Accessibilty Services at Southern Connecticut State University, for the employment and opportunity to develop these materials. I would like to extend this thank you for all of the administrative support I have recieved to my boss Kathleen De Oliveira, and coordinaters Tanisha Guadalupe De Jésus, Calleigh Cotter, and Mia Grella. Of course, I would like to extend this thank you to Professor Carl Haberfeld, for allowing me to attend his CSC-212 lectures.

2 Introduction's Introduction

I am very excited to dive into the wonderful world of **data structures**. Before we get started, there is just a bit of housekeeping to do. These matters are more relevant to the themes of this work, and the text as it appears on paper. I figure this is not really an introduction to data structures, its more of a introduction to data structures's introduction!

2.1 Python Examples

Topics and key ideas will usually correspond to some kind of Python example. An assumption is being made that the reader is to some extent *aquainted* with Python. There is absolutely no need to be a wizard of Pythonism, so there are notes clarifying syntax where necessary.

2.1.1 I Hate Snake Case

Per the title of this soap-box, I am not a fan of 'snake_case' as a naming convention. For the Python examples I will be writing in 'camelCase.' Is it my early Java Indoctrination peaking through? Maybe—but a set naming convention is almost entirely superfluous, and it looks nicer to my eyes. camelCase it is.

Let's get accustomed to it, here's an Example!

(`Example01.py`)

```python
def sayHi():
    print("Hello world!")

sayHi()
```

The reference to the actual script 'Example01.py' is included for your convenience, it can be found in the corresponding GitHub Repository!

2.1.2 Further Reading

Many of the terms and concepts featured in here will be based on the textbook *Problem Solving with Algorithms and Data Structures using Python* by Brad Miller and David Ranum. They have a fantastic online resource, so please check it out!

- https://runestone.academy/ns/books/published/pythonds/index.html

3 Introduction to Data Structures

3.1 Defining Data

Let's begin with getting the terms out of the way, and associating them with a defintion.

- **Atomic data types**: A datatype which may not be broken down into smaller parts.
- **Abstract data type** (ADT): A logical, abstract definition of how data is to be viewed.
-
- **Primative data type**: A definition of data, with regards to allowable values, and operations.
- **Data structure**: The actual implementation of an ADT in a given programming language.

These terms at first glance seem like sticking 'data' next to a bunch of other computer-y words. Some of their coonnections and relations can be muddy. So let's take a moment to try to understand them through some explanation, and examples.

3.1.1 Atomic Data Types

Atomic data types are types such as *integers*, *floating point numbers*, and *booleans*. Since these types do not get broken down into smaller pieces of data. Note, this is somewhat language-dependent. For example, Python treats it's boolean type bool as a subclass of int, and therefore it is not internally atomic.

3.1.2 Abstract Data Types

Abstract data types are instead defined by *behavior*, what it does. We will look into a data structure called a **Stack**. For now, know that this is a collection, and we define it as a collection which releases objects in reverse-order of their arrival. Note: I did not say this was a collection of any kind of thing, or had any specific operations. For ADTs, we're concerned with how they act!

3.1.3 Primative Data Types

The **primative data types** include data such as *integers*, *floating point numbers* and *booleans*. As given in the above definition, they are defined by the possible values and the possible operations they can participate in. These are used both on their own, but also in the further creation of data structures.

3.1.4 Data Structures

Data structures are the real deal. A data structure is the actual implementation of an ADT which we will interface with. Back to our stack, we might use a standard python list, and only append and pop items. This is a very basic example, with some data-structures later, we will be giving it more effort than this!

But with those definitions, lits now take a look at a python script, and identify what-is-what:

(Example02.py)

```python
# Primative Data Type
print("Im an integer, called ", type(6))

# Abstract Data Type
print("I want to release data in reverse order of it's arrival")

# Data Structure
stack = [1, 2, 3]
stack.append(4)
print(f"4 comes and goes: {stack.pop()}, leaving {stack}")
```

- An integer or floating point number are primative data types.
- ADTs are not the implementation, just the thoughts.
- We can implement a stack with a list, and treat it like one.

3.2 Moving Forward

I will use these terms accordingly. For basic components, I'll say "data type." "ADT" will be used when discussing the abstract way we're thinking of different kinds of data. "Data type/data structure" will be used when referencing a specific implementation, in python or otherwise. I will also use "data structure" to talk about the subject as a whole.

4 Basic Data Types and Structures

4.1 Integer, Floating Point Numbers, Booleans

4.1.1 Integer

The ADT integer can take a value from the set :$\{-n, ..., 1, 0, 1, ..., n\}$. Almost certainly your machine cannot hold the highest conceivable n, but remember: we don't care about that, were thinking in the abstract!

Anyway, along side those values, we can imagine the kind of operations that get performed on integers. We add them, subtract them, multiply them, and perform *integer division* and *modulo division*. I did not skip over "division" since we can't always get away with it in the abstract world. $2 \div 3$ is a no go! Let's investigate this further:

(Example03.py)

```
numOne = 1
numTwo = 2

#Check Types
print(type(numOne))
print(type(numTwo))

print(type(numOne + numTwo)) # Addition
print(type(numOne - numTwo)) # Subtraction
print(type(numOne * numTwo)) # Multiplication
print(type(numOne // numTwo)) # Integer division
print(type(numOne % numTwo)) # Modulo division

print(type(numOne / numTwo)) # Devision XX
```

Run this script, and notice that after all of these operations, they are all integers, (Python's 'int' data type). We'll need something a little more robust.

4.1.2 Floating Point Number

The floating point ADT is a number that has an integer part, as well as a fractional part. The type of values a floating point may hold is now: $-n, n$. Once again, this is not with regard to any implementation, so we define this interval as continuous. Actual computer numbers are not continuous, famously.

Feel free to change the initial variables numOne and numTwo to be Python's 'float' type, to get a sense of the operations. All of them are the same, with the added benefit of addition.

4.1.3 Boolean

Booleans as an ADT can take the value of either "$True$" or "$False$." The kind of operations that they can undergo are taken directly from boolean logic (naturally). This includes AND, OR, and NOT. I will spare the details of

logical operations here, more information on logic can be found in another one of my lecture notes on Computer Systems.

4.2 Collection Types

We'll keep this conversation in the abstract, without getting deep into any one implementation of these data structures.

4.2.1 Strings

We'll think of a string as a collection of characters. these characters are ordered in so far as the way they are written. We can compare strings, splice strings, concatenate them, etc.

4.2.2 Lists

Lists are like an even more abstract version of strings. While they can be ordered, they definitely don't need to be.

Lists can be indexed, they can be spliced. You can add things and remove things from lists. Tt does not truly matter *what* you store inside of them! We'll dive into the types of lists later on.

4.2.3 Maps

Maps or **Dictionaries** (it is not worth getting stuck in the mud of one language's keyword) are collection types defined by a key-value pair.

Think about it this way, when you go to the bank, and enter your pin number to access your count, what does the bank say:

1. "Yup, definitely your pin number!"
2. "Ah, here's your account information!"

The second one, but why? Well, (for simplicities sake) your pin number is a kind of key that lets the bank look up your account information, and give it to you. If we didn't have this way to map information to each other, we'd have a lot more replies like the first response.

4.2.4 Miscellaneous

We also have **sets** and **tuples**. I do not see these being potent topics for this course, so I will let mentioning them suffice. I encourage you to look further into them!

4.3 Moving forward

We've covered probabily the most encountered ones, with enough detail that will get accross the broad themes (hopefully mostly a reminder from your introductory course on programming). Many of the remaining collection ADTs warrant their own Section!

5 Analysis

5.1 Analyzing Algorithms

What do we mean when we say we want to analyze an algorithm? Basically, we are trying to quantify algorithmic performance to compare algorithms, and understand which ones perform better than others, and perhaps what makes them better. For our purposes, analysis will pertain specifically to **runtime**, or the time it takes for an algorithm to output a solution.

But how do we analyze this? Well, there are two ways. One, is a **theoretical analysis**. I'll refer to an algorithms theoretical performance as **running time**. The other way is to actually implement, and run the algorithm in a **benchmark**. In this manner, I will refer to an algorithm's performance as **runtime**. Confusing, I know, but it will give us a concrete way to distinguish theory from implementation.

5.2 Benchmarking

We'll begin by exploring benchmarking, since this is the most straightforward concept. Imagine you were trying to understand your athletic performance. You start your watch, run a few laps, and then stop the clock. From the distance you traveled, and the time you recorded, you would be able to determine a kind of pace.

We do the same for algorithms! First, we'll get the system's time, run the algorithm, get the system time once more to determine the total time it took to run.

(Example04.py)

```python
import time

def algorithmOne(n):
    for i in range(n):
        for j in range(n):
            x = i + j

n = 100

startTime = time.perf_counter()
algorithmOne(n)
stopTime = time.perf_counter()

totalTime = stopTime - startTime
print(totalTime)
```

This is exactly what we do here! This outputs a time! However, this is a very barebones framework, we likely want to track it. Let's beef this code up, and see how we can compare performance.

Let's introduce a theme here: *we want to know how the algorithm performs as the input increases in size*. We get more helpful information, by running multiple trials with increasing input size.

Another way to improve a benchmarking experiment is to repeat trials, and record an average of the performance. One pitfall of benchmarking (we'll return to this idea) is that it is ultimately system dependent. Having other programs running on your computer will affect a benchmarking performance. This is unavoidable, but it can be soothed by running multiple trials with identical conditions, and averaging them out.

(Example05.py)

```python
import time

def algorithmOne(n):
    for i in range(n):
        for j in range(n):
            x = i + j

def benchmark(trials, runs):
    n = 1000
    for i in range(trials): # Trial

        times = []
        for j in range(runs): # Repeat identical trials
            input = n*i
            startTime = time.perf_counter()
            algorithmOne(input) # Increaseing input,
            stopTime = time.perf_counter()

            times.append(stopTime - startTime)

        print(f"Trial: {i + 1} -- input: {input} -- Average performance: {sum(times)/len(times)} seconds")

benchmark(5, 5)
```

Now this looks like a benchmarking experiment! But we can do better. Let's add some graphics! It will help us to paint a picture (iterally) of perfomance. I encourage you to check that out! I have put the output in Figure 1. Please note that this graph looks an awful lot like the graph $f(x) = x^2$ (cause it is), and we will see more on this in the next section!

5.3 Theoretical Analysis

As we stated earlier, we are somewhat restricted by the environment we run experiemnts in. So far, we've been benchmarking Python functions, but really these functions are algorithms, that just happen to be in Python. The broader thing, the algorithm, is not beholden to any particular computing envorionment.

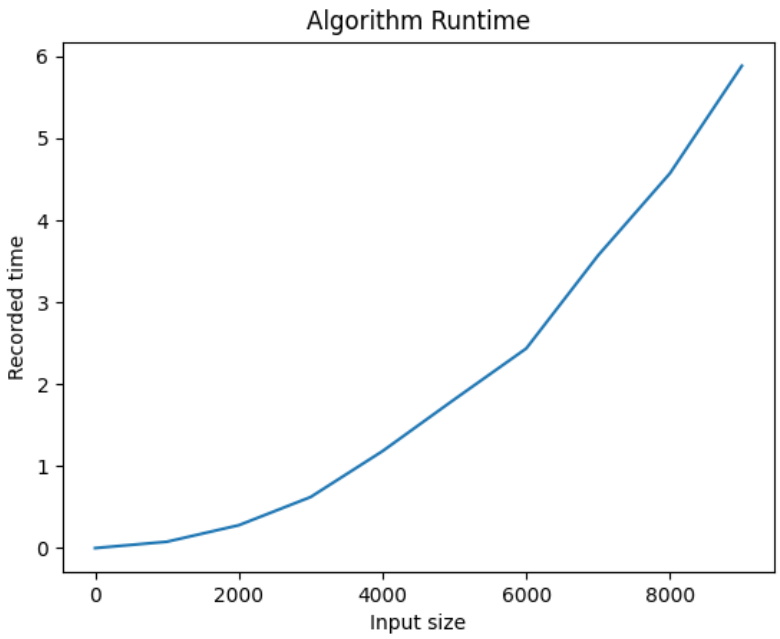

Figure 1: featured in `Example06.py`

Because of this, we want a way to analyze it independent of a computing environment. That is what we call, *algorithm analysis*. This topic is incredibly deep and very interesting, and you will almost certainly take a course on algorithm analysis down the road. We introduce the topics and themes when learning about data structures!

5.3.1 Intuition

Let's use this kind of intuition: we are counting the amount of work that an algorithm does. However, we are never concerned with a finite input, we are worried about an infinite input!

Think about it this way, how does the size of the task change the amount of time it takes to complete? Remember in the above example, we increased the work by a fixed (linear) increment. But the work each time took more and more each time!

5.3.2 Unit of work

We are going to be counting "Units of work" this is essentially any sort of computation-intensive process in an algorithm. We'll be using variable assignments as our unit of work. Think about it, in the long term, the amount of

compute that goes towards adding two integers will not rival the amount of work needed to actually store the result in a variable in memory (remember think long term)!

5.3.3 Worst-case

We also want to think about things in *worst-case* scenarios. Not only are we considering an input which is arbitrarily large, we are considering the situation when this input is so twisted, the problem couldn't be any harder!

Say you have a list, and you want to sort it. Well if you input a list that is already sorted, that's easy! Because it's easy, it doesn't really capture how an alogrihtm performs.

But what if you input a list in reverse order? That is going to be tough for an algorithm. We think about situations like this.

5.3.4 Analysis and Notation

Let's refer back to the function in *Exmaple04.py*. This of course is in python, but forget the Python! think of this as some algorithm that happens to be in Python.

We see that this algorithm takes an input n. So, we will write a *running time function* $T(n)$ which will describe All of the wok that is going on.

First, we'll jump all the way into the inner-most loop. We see that we are doing $i + j$, and then storing that result in x. Again, we're not so worried with how much work goes into performing $i + j$, we are only concerned with the work that goes into x. We'll simply count this as 1.

The question is, how many times do we perform this 1 unit of work? To determine this, we look at the loop this operation belongs in, and we see that this is done n times, the same exact n as our input! We model this as follows:

$$T(n) = (n \times 1)$$

But we are not done, becasue that loop is inside another loop, also controlled by n! so our $(n+1)$ happens n times, and we will model that with the following:

$$T(n) = n \times (n \times 1)$$

A little bit of algebra, and we determine the running time function of this algorithm to be:

$$T(n) = n^2$$

5.3.5 Big-O

Now, of course when a variable coefficient is 1, it is unecessary. But say our runningtime functoin instead looked something like this, where c is an integer that **does not change** (is constant). Something Like this:

$$T(n) = cn =^2$$

Well, to determine the running time, we drop such constants. We call this the **Big-O**. Think about it, if our input n is arbitrarily large, then in some time, it will be bigger than a constant value c (whatever c is, n could be $c+1$, but in realaity its infinitely larger). So we drop it, since it will bever compare to the arbitrarily large input. With that, we say that this algorithm runs in:

$$O(n^2)$$

"O" is for "Big-O" and the stuff inside the parenthesis is the asymptotically significant portion of the runningtime function (notice no cs).

6 Linear Structures

Linear structures are data structures that utilize some kind of linear orientation. This involves two ends, and a sequence of intermediary parts between these two ends. Each part is connected to the part in front of it, and the part behind it (if it exists). These can be thought of as collections where the elements maintain some kind of ordered relationship like this, and we will see the basic ADTs that have linear properties.

When we go to implement data structures, especially in Python, most of these will be list-based.

6.1 Stacks

The key Idea to remember for Stacks is the **LIFO** principle, or the "*last in first out*" principle. Items are added and removed exclusively from the top of the stack, building off of the base. If you would like to add items, it should be done directly on top. If you would like to remove items, the last item should be taken of first.

Think about this like a literal stack of blocks. If you stack them by block a, then b, then c, if you would like to remove block a, then b must certainly should come off first, otherwise the whole tower wil topple! Similarly, if you want to remove block b, then c must be removed first. This can be visualized in Figure 2.

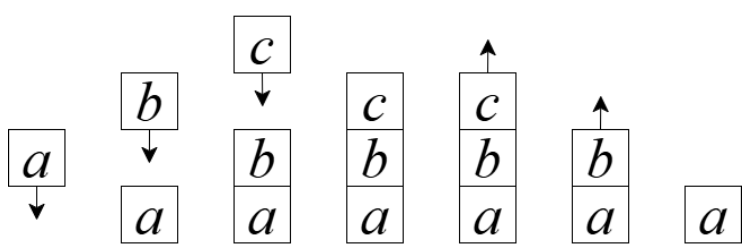

Figure 2: Stack of blocks a, b, and c

6.1.1 Stack implementation

We would like to have the following methods in our stack:

- **Initialize**: Initialize a stack object.
- **Push**: We want to add an element onto the top of the stack.
- **Peek**: We would like to see the last element without removing it from the stack.
- **Pop**: We would like to grab the last element, removing it from the stack.

- **isEmpty**: True if our stack contains at least one element, otherwise False.
- **Size**: We would like to know how many elements are pushed onto our stack.

In Python, we will be implementing a stack by defining a class called 'Stack', and adding all of the desired functionalities. As can be seen, this is largely based off of the natural list methods in Python. It is fairly standard for stacks to be 'implemented' just as lists, with code that treats them with the "stack rules." For the sake of learning however, we will be implementing a stack from a completely object-oriented perspective.

(Example07.py)

```python
class Stack:

    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def peek(self):
        return self.items[-1]

    def pop(self):
        return self.items.pop()

    def isEmpty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

S = Stack()
print(S.isEmpty())
S.push('a');S.push('b');S.push('c')
print(S.isEmpty())
print(S.size());print(S.pop());print(S.size())
```

Here is the Implementation. Please play around with this, I have already left some code using some stack methods!

6.1.2 Stack Analysis

The stack operations **Pop** and **Push** run in $O(1)$, since no matter how large the stack becomes, these methods will only ever work at the end, and thus can be accessed in constant time.

Similarly with **isEmpty**, only the first element needs to be checked. It does not matter how many items are in the stack, as long as there is one, it returns False. Of course, if there are no items, it can just as easily return True.

Size however, runs in $O(n)$ time. This is becasuse this method will have to pass every single element exactly once. In the *worst case*, when you have n elements, it will take $O(n)$ time.

Use these kinds of reasoning to think about the runtime of the other methods!

6.2 Queue

The queue operates slightly differently than the stack, operating with the **FIFO** principle, or "*first in first out*" Principle. Still a linear layout, but instead of having a base and a top like the stack, it has a front and a rear. Items are added exclusively to the rear, and are removed exclusively from the front. An example using blocks can be seen in Figure 3.

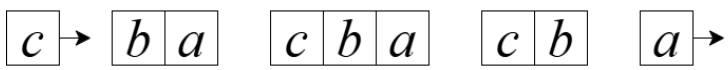

Figure 3: Queue of blocks a, b, and c

Think of this like a line at a cafe. You can only enter the line at the very end, and until you can reach the front to place an order, all preceding line members must exit from the front of the line! Perhaps this is obvious, but service lines are often simply called queues!

6.3 Queue Implementation

To implement a Queue, we would like the following methods:

- **Initialize**: Initialize a queue object.
- **Enqueue**: Add an element to the queue.
- **Dequeue**: Remove an element from the queue.
- **isEmpty**: (See Stack)
- **Size**: (See Stack)

This can be done by the following code

(Example08.py)

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.insert(0, item)

    def dequeue(self):
        return self.items.pop()
```

```python

    def isEmpty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)
```

6.3.1 Queue Analysis

The runningtimes and their justifications are similar to the stack, with the exception of **enqueue**. this is because a new item is inserted at the front. To do this, an entire list must me shifted to the right, to allow space for the new item. You guessed it, for a list containing n items, inserting at the front would require n shifts, thus running in $O(n)$ time.

6.4 Deque

A deque comes from the term *double-ended queue*, which should be insightful into it's kind of behavior. It acts exactly as a queue, but items can be inserted and removed from either end.

The following are the must-have methods for creating a dequeue:

- **Initialize**: Initialize a deque object.
- **AddFront**: Add an item to the front.
- **AddRear**: Add an item to the rear.
- **RemoveFront**: Remove an item from the front.
- **RemoveRear**: Remove an item from the rear.

6.4.1 Deque Implementation

C'mon, you can do the implementation!

6.4.2 Deque Analysis

Our analysis of queue methods will help us for working with the deque. Methods that utilize the rear will run in $O(n)$ for the same reasoning as simply enqueueing an element did. Likewise for methods involving the front, this can be immediately accessed at the end, which is independent of deque size, meaning they work in $O(1)$ time.

7 Searching and Sorting

Two pivotal domains of computer science stem from both **searching** and **sorting**. For our application, we might be more concerned with the kind of methods we code into the data structures that we create.

For instance, when you index a list in Python, the '[]' invokes an index method, that retrieves the item at the index you have specified.

We will examine searching and sorting algorithms, and do some analysis!

7.1 Searching

Searching involves looking for a particular element within a collection. If the item is in the collection, we should see a "True" or a "False" if the item is not in the collection.

7.1.1 Sequential Search

This involves checking each item one by one down the colleciton, seeing if the item we are looking for matches the one we are currently on.

For example, say we have the list: $[1, 2, 3, 4, 5]$ and we are looking for 3. We would check:

$$\begin{array}{lll} [\mathbf{1}, 2, 3, 4, 5] & 1 == 3? & \text{no} \\ [1, \mathbf{2}, 3, 4, 5] & 2 == 3? & \text{no} \\ [1, 2, \mathbf{3}, 4, 5] & 3 == 3? & \text{yes} \rightarrow \text{True} \end{array}$$

How do we analyze this? Well, we start be considering the worst case: the collection doesn't even contain the item we are looking for! Since this algorithm is sequential, we are going to 'hop' over every single element of the input, as it will never be stopped.

Say at every 'hop' we do a single unit of work. If we input an arbitrarily large input collection with n elements, we are going to n units of work in the worst case. Therefore, this algorithm runs in $O(n)$ time.

7.1.2 Binary Search

Sequential search is indeed necessary for an unsorted collection. However, if we know that our input collection is in fact sorted, we can get away with a faster search algorithm!

Binary search utilizes a collection which is sorted. It works in different *levels*. At each level, we ask: "Is the item we are looking for less than, equal to, or greater than the element located in the middle of the collection? If it is less than, the item in the middle, then we only need to consider the left half of the list. Similarly, if the item we are looking for is larger than the middle item, we only need the right side of the list. And of course, if our item equals the middle one, we've simply found it!

So, back to our worst-case analysis. We have an arbitrarily large collection with n sorted items, and we are looking for an item in it. Once again, in the worst case the item will not actually be in our list.

So at level 0, we are looking at n elements. But then, at level 1, our list splits in half, now focusing on $n/2$ elements. at level 2, we once again split in half and are focused on $n/4$ elements. In fact, this continues, and for an input list with n elements, we consider $n/2^l$, where l is the number of the level.

At each level l, there is one unit of work. So for an input size of n, there are l units of work. This can be solved for to obtain $l = \log n$. Therefore, the running time of Binary Sort is $O(\log n)$.

7.2 Sorting

We mentioned an input list being *sorted*, and being the very thing that makes the binary search more efficient. What does that mean?

Sorting is the task of taking a collection, and arranging it in a particular manner. As we proceeed, consider "sorted" to be by numercal value, where the first item in the collection is the smallest value, and the last item is the largest value.

7.2.1 Bubble Sort

Bubble sort works through several passes. At each step, the algorithm will look at an item, and the item immediately after it. If these items are out of place (The first item is larger than the second) then it will swap the two. The term 'bubble' comes from the way that values tend to bubble up toward the back of the lsit through each iteration.

Let's shift our focus to time complexity. As always, we have a collection of n elements. The worst case scenario for a sorting problem is that the collection is input in the completely reverse sorted order! We will move forward with this as our worst case input.

Every single iteration, the Bubble sort is going to make a swap, i.e. perform a unit of work at every step. Remember, the collectiom is in reverse order, so the first item is indeed larger than every proceeding item. The first item c is larger than the other $n-1$ elements. Therefore, $n-1$ swaps will be made, or $n-1$ units of work will be performed.

In the next iteration, only one item is sorted. We can think of this as the exact same problem (since the rest of the collection is still in reverse order). Now in total, it will perform $n-2$ units of work. This will continue for each iteration until 1 unit of work is performed, and then our collection will be sorted. This will require $n-1$ passes!

We are making $(n-1)$ comparisons $(n-1)$ times, which we can call $(n-1)^2$, resulting in a time complexity of $O(n)$.

7.2.2 Merge Sort

We're skipping some of the other big sorting methods, and jumpring right into Merge Sort. Merge sort utilizes a method known as **recursion**. If you know what recursion is, great! If not... also great! We'll explain it without getting too nitty-gritty into the recursivity of Merge Sort. If anything, this might help to build some intuition on recursion in general!

Merge sort works by taking an input collection, and splitting it up, and then splitting that, and splitting that, etc., until we end up with many collections each containing exactly one element. From there, we *merge* the collections so that they are sorted. Here's a general Idea. You can imagine that the input collection is unsorted, and then at each level, the 'sub-collections' are put back together so that they are sorted!

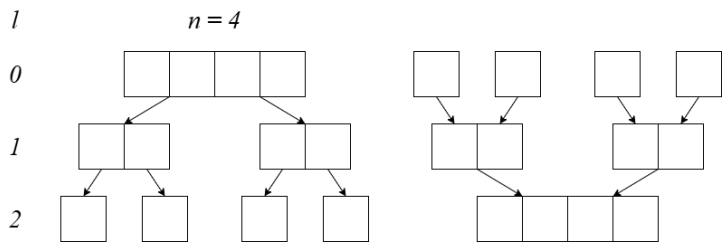

Figure 4: Merge Sort Operation

To analyze this, we need to understand two things: 1. How many times do we split, the input collection. Then 2. what is the complexity of doing so? We then need to know the complexity of merging two lists.

1. Splitting a collection.

   Splitting a list into two lists happens in O(1) constant time, we will consider this a unit of work. how many times do we split our collection? We'll pull our thinking right from Binary sort, where we split in half, and half, and half. So we know that for an in put of n, we end up with $\log n$ levels.

2. Merging two collections

   Next, we need to merge two collections into a sorted manner. We have two collections, of size n and m. In the worst case scenario, we must check every element in both collection, and add it to the merged collection, which is one unit of work. Meaning that for a given merge, we perform $n + m$ units of work.

Now we're ready to analyze Merge sort. Consider an input collection of size n. Then, consider any given level, we will perform any number of merge operations, which we described to be $n + m$. However, we know that at each level, the *total* number of elements across all of the merge operations is n. so, we in fact perform n units of work! As described, we need $\log n$ levels. This

means that we perform $n \times \log n$ units of work, meaning Merge sort runs in $O(n \log n)$ time.

8 Trees

Fundamentally, **trees** are a type of *graph* thats *connected*, and *acyclic*. If that sounds like a lot of jargon, then that's okay! This is the definition from a graph theoretic standpoint. I am beginning with this to get across the idea that trees truly are that abstract, and you'll see them everywhere!

We don't need to stand on this definition though! We'll define some terms below, and continue with them, in a more data structure-y way, than a graph theory-y way. You will learn much more about that in discrete math class, and algorithms class!

8.1 Terms

- **Node**: These are the building blocks of trees, these are subjects that will share some kind of relation.
- **Edge**: These connect nodes.
- **Root**: The *root* of a tree is the single starting point, usually depicted at the very top, from other nodes follow.
- **Children**: In a tree, every node that is connected "beneath" another node, is a *child* of the top node. The *children* of a node is the collection of *child* nodes connected to it.
- **Parent**: The *parent* of a child node, is the node that a child comes from.
- **Leaf**: A leaf node is a node in the tree that has no children.
- **Level**: The *level* that a node belongs to is the number of nodes that need to be traversed to get to it.
- **Height**: The *height* of a tree is the same as the largest level in the tree.
- **Subtree**: A tree in which the root node is a child of another node.

8.2 Types of Trees

Unless a contradiction is obvious, note that many of these tree type's are more qualities, and a given tree may adhere to one or more of these!

8.2.1 n-ary Trees

These are trees where a node can contain any number (n) children.

8.2.2 Binary Trees

These trees are trees where each node has exactly 0, 1, or two children. An additional property holds that for any given node with value i, the value of every node in the left subtree is less than i, and the value of every node in the right subtree is greater than i.

8.2.3 Balanced Trees

A Balanced Tree is a tree where the depth of every node equals the height h, or $h - 1$. This is often a property that *binary search trees* maintain.

8.3 Binary Search Trees

Binary Search Trees (BSTs) are essential data structures, as they strike a compromise between fast lookup and storage. We can store sortable items in nodes, in which the properties of the BST will naturally sort them.

8.3.1 Lookup

Say we want to lookup an item. Well, we just compare it to the root node to see if they are the same. If it is different, and less than the root node, we will traverse left, otherwise we will traverse right. Now we arrive at a new node where we will repeat this process!

When A BST is balanced, this gives us $O(\log n)$ lookups! Since for a balanced tree of n items, we double the nodes in each increasing level, meaning we have $\log n$ levels. In the worst case, we only have to touch $\log n$ nodes! Because this is such great searching, sometimes some extra work is done up front to ensure a BST is balanced at all times!

If it is not balanced however, lookups do indeed become a $O(n)$ operation. Consider a root which is the largest (or smallest) in the tree, where every node has only a left subtree, and is the subtree of another node. In the worst case we are looking for the value that is the lowest, and we would need to hit every single one! But oftentimes, in the *average case* lookups are still $O(\log n)$.

References

[1] Bradley N. Miller and David L. Ranum. *Problem Solving with Algorithms and Data Structures Using Python*. Franklin, Beedle & Associates, Wilsonville, OR, 2 edition, 2011.